

Diseño e Implementación de Agentes Inteligentes basados en LLMs

Grupo Nro 4

Mateo Gabriel Blanche - mateoblanche5@gmail.com

José Carlos Cammisi - joseccammisi@gmail.com

Valentino Delarmelina - valentinodelarmelina@outlook.com

Franco Yucci - francoycc@gmail.com

Resumen. Este trabajo presenta el diseño conceptual y la implementación de un agente inteligente autónomo basado en Modelos de Lenguaje (LLM) encargado de asistir en el proceso de *onboarding* (inducción) corporativa y mentoría para nuevos empleados. El problema central que resuelve es la sobrecarga de información desestructurada al ingresar a una organización, facilitando la adaptación del empleado mediante lenguaje natural (vía texto o audio transcrito). El sistema integra un orquestador que ejecuta un bucle de razonamiento ReAct, conectándose dinámicamente mediante *function calling* a la API de Google Calendar para coordinar bloques de estudio y reuniones de seguimiento sin superposiciones (*overlapping*). Como factor diferenciador, el agente incorpora un módulo RAG (Retrieval-Augmented Generation) sobre una base de datos vectorial que almacena manuales corporativos, políticas de RRHH y rutas de aprendizaje (*learning paths*), garantizando una asistencia contextualizada, además de un sistema de memoria conversacional para interacciones multi-turno.

1. Introducción

En la actualidad, el proceso de inducción (onboarding) de nuevos empleados, especialmente en perfiles Junior, suele representar un cuello de botella en las empresas. Los nuevos ingresos se enfrentan a una abrumadora cantidad de información desestructurada (manuales de cultura, políticas de licencias, configuraciones técnicas y planes de capacitación) que dificulta su rápida adaptación. Esto genera interrupciones constantes a los equipos de Recursos Humanos y a los desarrolladores Senior, retrasando la productividad. Con el auge de las interfaces en lenguaje natural impulsadas por IA Generativa, surge el área de aplicación de los Agentes Inteligentes Autónomos como mentores corporativos capaces de guiar este proceso y actuar sobre el entorno digital del empleado.

El problema concreto a resolver en este proyecto es el diseño de un agente asistente que interprete consultas e intenciones ambiguas de empleados recién

ingresados (por ejemplo: "Soy dev junior, me llamo Valentino, ¿qué tecnologías tengo que aprender esta semana y cuándo me puedo reunir con mi líder?"). El agente debe utilizar técnicas de Recuperación Aumentada por Generación (RAG) para extraer semánticamente las directrices exactas desde los documentos corporativos, traducir esas necesidades a parámetros estructurados de tiempo, y usar herramientas externas para modificar el mundo real (Google Calendar). La principal restricción de diseño es que el agente no debe agendar a ciegas: si existe un evento preexistente que colisione con los nuevos bloques de estudio o reuniones, el agente debe frenar la acción, advertir al usuario y calcular dinámicamente un nuevo espacio disponible. El flujo básico de interacción del sistema propuesto se puede observar gráficamente en la Figura 1.

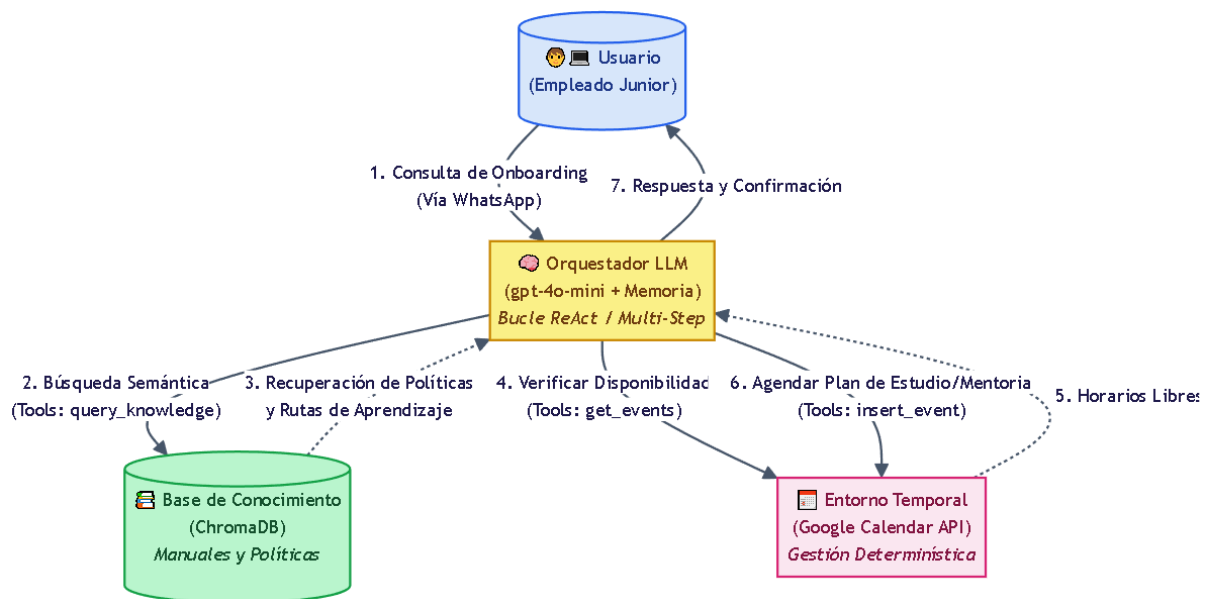


Figura 1. Esquema general de interacción entre el usuario, el orquestador basado en LLM, la base de conocimiento documental y el entorno de Google Calendar.

Cabe destacar que este diseño supera el concepto de un chatbot pasivo. El agente implementa un bucle interactivo secuencial (Multi-Step) para gestionar la planificación: ante solicitudes como "Organizame mi primer día de trabajo", el sistema no se limita a responder con un listado de consejos; en su lugar, analiza las políticas recuperadas por el RAG (ej. "leer código de conducta"), consulta la disponibilidad real en la API de manera determinística, detecta posibles conflictos, presenta al empleado los bloques de tiempo propuestos y solicita una confirmación interactiva mediante su memoria conversacional antes de efectuar la escritura final en la agenda.

En la sección 2 se detalla la solución conceptual del agente, describiendo su arquitectura, el sistema de prompts, el bucle de razonamiento elegido y la integración de las herramientas junto al RAG. En la sección 3 se proponen los escenarios para las pruebas y la captura de resultados. Finalmente, en la sección 4

se exponen las conclusiones y las líneas de desarrollo futuras.

2. Solución

Objetivo del Agente: Asistir de forma autónoma en el proceso de inducción (*onboarding*) y capacitación de nuevos empleados a partir de solicitudes en lenguaje natural (texto o transcripción de audio). El agente debe resolver consultas sobre cultura organizacional y normativas extrayendo información semántica de manuales corporativos (mediante RAG) y, a su vez, gestionar proactivamente la agenda del usuario en Google Calendar para organizar bloques de estudio obligatorios y reuniones de mentoría y avances, garantizando la ausencia de conflictos horarios y proponiendo alternativas si se detecta una superposición.

Clasificación del Ambiente:

Parcialmente observable: El agente no conoce de antemano el estado de la agenda del usuario ni posee en su memoria base los extensos reglamentos y planes de carrera de la empresa; debe descubrirlos consultando activamente la API y la base vectorial.

Secuencial: Cada decisión de agendado, respuesta normativa o resolución de conflictos depende fuertemente de las interacciones previas y del progreso del empleado en su ruta de aprendizaje.

Dinámico: El entorno puede modificarse externamente (por ejemplo, si entra un correo o una invitación de calendario externa en paralelo mientras el agente está calculando su respuesta).

Estratégico / Estocástico: El comportamiento humano y la disponibilidad de otras personas (ej. un líder técnico para una mentoría) introducen incertidumbre sobre el éxito final de la cita.

Discreto: El entorno temporal se organiza mediante unidades preestablecidas. Los turnos, reuniones y bloques de capacitación ocupan franjas delimitadas, permitiendo al agente identificarlas con exactitud matemática.

Percepciones:

1. Mensajes de texto del usuario o transcripción del mensaje de voz (interfaz).
2. El estado actual de la agenda obtenido en formato JSON desde la API de Google Calendar.
3. Fragmentos de texto (chunks) recuperados desde la base de datos documental (ChromaDB) con las normativas corporativas.

Para resolver el problema del onboarding predictivo y autónomo, diseñaremos un agente inteligente basado en las capacidades de razonamiento lógico y ejecución de herramientas de un LLM. El Entorno es un ecosistema digital compuesto por las cuentas de Google Calendar, la base vectorial de la empresa y la interfaz de usuario web (WhatsApp).

Con el fin de robustecer el marco teórico resulta fundamental formalizar las especificaciones del agente mediante el modelo PEAS sustentado en la bibliografía de Russell & Norvig :

- **Medidas de Rendimiento (Performance):** Precisión semántica en la recuperación y explicación de políticas internas, fidelidad en la asignación de planes de estudio según el *seniority* del empleado, eficacia en la detección de *overlapping* temporal, y tasa de éxito en interacciones multi-turno sin perder el contexto.
- **Ambiente (Environment):** Entorno digital integrado por los servicios de Google Calendar, la base documental de la empresa y la interfaz de WhatsApp.
- **Actuadores (Actuators):** Conjunto de herramientas (*Tools*) que el orquestador puede ejecutar mediante *function calling* (búsqueda en base vectorial y peticiones REST a Google).
- **Sensores (Sensors):** Entradas de texto/audio del usuario, *payloads* de respuesta de la API de Google, vectores de contexto recuperados e historial de diálogo indexado en la memoria.

a. Arquitectura General y Flujo de Control

La arquitectura del agente sigue un esquema centralizado (Tier 1) donde el LLM actúa como el "cerebro" orquestador. El flujo se compone de tres capas principales: la capa de Interfaz, la capa de Razonamiento (el LLM interactuando con su memoria de corto plazo) y la capa de Integración (el conector con las APIs y la base RAG). El flujo de control sigue el principio de separación de responsabilidades: la extracción de conocimiento normativo es semántica (RAG), mientras que la planificación de eventos es estrictamente matemática y determinística (API Calendar).

b. System Prompt y Estrategia de Prompting

El *System Prompt* define el rol corporativo, restricciones y el flujo lógico obligatorio:

"Actuás como un Mentor y Asistente de Onboarding corporativo. Tu objetivo es guiar al nuevo empleado en su adaptación y organizar su tiempo. Reglas estrictas:

1) Ante dudas sobre qué debe hacer, qué debe estudiar o políticas de la empresa, siempre debés consultar la base de conocimiento corporativa (RAG).

- 2) *Antes de crear cualquier bloque de estudio o reunión en el calendario, tenés la obligación absoluta de leer los eventos previos mediante tus herramientas para verificar disponibilidad real.*
- 3) *Si detectás un conflicto de horarios, no debés crear el evento; debés proponer proactivamente alternativas disponibles utilizando la memoria del canal. "*

c. Tools / Function Calling

El agente cuenta con un catálogo de funciones estructuradas que el LLM puede invocar mediante *Function Calling*. Cada herramienta cuenta con un esquema JSON rígido que define sus entradas:

- `query_company_knowledge(query: string)`: Realiza una búsqueda semántica (RAG) en la base de datos local extrayendo directrices de manuales de RRHH, cultura institucional y rutas de aprendizaje tecnológico.
- `get_calendar_events(start_time, end_time)`: Lee de manera determinística los compromisos existentes en un rango horario.
- `insert_calendar_event(title, start_time, end_time, description)`: Registra el nuevo evento en Google Calendar.
- `find_available_slots(date, duration_minutes)`: Herramienta auxiliar de conveniencia que analiza la jornada y devuelve bloques libres.
- `query_user_profile(query)`: Recupera las restricciones semánticas de la rutina del usuario mediante embeddings.

Validaciones y Errores: Si el usuario ingresa datos parciales, el LLM repregunta antes de llamar a cualquier tool. Si la API de Google falla (error de token) o ChromaDB no encuentra documentos relevantes, la capa de código captura la excepción y se la pasa como texto al LLM para que este se la explique al usuario amigablemente.

d. Loop del Agente

Se seleccionó el framework de razonamiento ReAct (Reasoning and Acting) sobre grafos de estado. La justificación de esta elección se basa en que nuestro ambiente es parcialmente observable; el agente no sabe qué dicen los reglamentos ni qué hay en el calendario hasta que actúa. ReAct le permite alternar de forma orgánica entre:

1. Pensamiento (Thought): El usuario es un desarrollador Junior y me pide que le organice su primer día. Primero debo consultar el plan de onboarding en la base de conocimiento corporativa.
2. Acción (Action): Ejecutar `query_company_knowledge(query = "tareas obligatorias primer día desarrollador junior")`.
3. Observación (Observation): El RAG devuelve: "El empleado debe leer el código de conducta (1 hora) y realizar el tutorial de Git corporativo (2 horas)".

4. Pensamiento (Thought): "Ahora que conozco sus tareas obligatorias, debo leer la API de Google Calendar para encontrar 3 horas libres en su jornada y agendarlas."
5. Acción (Action): Ejecutar `get_calendar_events(...)`.
6. Observación (Observation): "De 09:00 a 13:00 está libre."
7. Acción (Action): Ejecutar `insert_calendar_event(...)` para bloquear la mañana de estudio.

e. Memoria

El agente maneja una memoria de Corto Plazo (Short-term Memory) gestionada dentro del estado del grafo (MessagesState). Almacena los últimos turnos de la conversación en formato de lista de mensajes [User, Assistant, ToolMessage, Assistant]. Esto permite llevar el hilo de la negociación del turno y de las consultas normativas.

f. *Guardrails y Validación*

Para evitar fallos catastróficos, se configuran las siguientes salvaguardas:

- Límite de Iteraciones (Max Loops): El ciclo ReAct tiene un techo estricto de 5 llamadas a herramientas por turno. Si el modelo entra en un bucle repetitivo, el sistema corta la ejecución.
- Manejo de Alucinaciones: Se prohíbe explícitamente inventar reglas corporativas que no hayan sido devueltas por la herramienta RAG, así como confirmar una reserva horaria sin haber recibido el output exitoso de `insert_calendar_event`.

g. *Observabilidad*

El sistema registrará el flujo de ejecución completo mediante trazas detalladas. Esto nos permitirá monitorear en tiempo real los pensamientos internos del modelo, las distancias de similitud de los vectores recuperados en el RAG, las funciones llamadas, y el consumo exacto de tokens para llevar un control estricto de costos.

La arquitectura propuesta para el asistente inteligente autónomo migra de un esquema puramente reactivo hacia un modelo híbrido proactivo y dirigido por objetivos. El sistema combina un Módulo de Perfilado Proactivo e Ingesta RAG con una Capa de Planificación Multi-Step implementada sobre grafos de estado. El diseño se fundamenta en desacoplar la percepción lingüística, el corpus documental de la organización, la persistencia de los hábitos del nuevo empleado, el razonamiento lógico secuencial y la ejecución física sobre el entorno digital.

El ciclo de vida del agente se divide operativamente en dos fases bien definidas:

Fase 1: Descubrimiento y Perfilado Semántico

El agente detecta mediante su flag de estado si se trata de la primera interacción con el nuevo ingreso. En lugar de esperar comandos, toma la iniciativa e inicia una entrevista interactiva guiada (preguntas sobre su rol, seniority, horarios de trabajo asignados y actividades fijas). El LLM procesa las respuestas desestructuradas, las consolida en un documento Markdown/JSON de restricciones de usuario y las guarda en el almacén vectorial para ser consultadas por la herramienta correspondiente.

Fase 2: Planificación Operativa Guiada por RAG

Ante una solicitud compleja o ambigua del empleado (ej. "Organizame la semana de inducción"), el agente ejecuta una planificación multi-paso. Primero realiza una búsqueda semántica dual: extrae las políticas del puesto y su ruta de aprendizaje técnico (Learning Paths) y recupera el perfil de rutina del usuario. Con este contexto indexado en la ventana de contexto, el planificador calcula matemáticamente los bloques temporales idóneos en la API de Google Calendar y ejecuta secuencialmente las escrituras.

2.1. Arquitectura General del Agente: Módulos, Componentes y Tecnologías

La arquitectura propuesta para el asistente inteligente autónomo migra de un esquema puramente reactivo hacia un modelo híbrido proactivo y dirigido por objetivos, combinando un Módulo de Perfilado Proactivo (RAG Dinámico) con una Capa de Planificación Multi-Step. El diseño se fundamenta en desacoplar la percepción lingüística, la base de conocimientos de estilo de vida del usuario, el razonamiento lógico secuencial y la ejecución física sobre el entorno digital.

A continuación se detallan los módulos funcionales, las tecnologías seleccionadas y los componentes internos que dan vida al sistema.

1. Módulos Funcionales del Sistema

A. Módulo de Interfaz y Percepción Lingüística (Capa de Entrada/Salida)

Este módulo actúa como el sensor primario del agente. Se encarga de la comunicación bidireccional con el usuario humano a través de canales asincrónicos.

- Componente de Recepción: Captura los estímulos entrantes (mensajes de texto o flujos de audio) enviados por el usuario a través de la pasarela de WhatsApp.
- Componente de Transcripción (STT - Speech-to-Text): Si la percepción recibida es un archivo de voz, el subcomponente procesa la señal de audio

mediante la API de OpenAI Whisper, traduciéndola a texto plano optimizado semánticamente para mitigar modismos regionales y ruidos de fondo.

B. Módulo de Perfilado Proactivo y Onboarding (Fase de Descubrimiento)

Responsable de romper la pasividad del chatbot tradicional.

- Lógica de Interrogación: Orquesta la entrevista interactiva inicial. Formula de manera secuencial preguntas clave sobre el rol, horarios de equipos asignados y flexibilidad del ingresante.
- Estructurador de Perfil: Toma la transcripción libre del diálogo de inducción, extrae las entidades temporales blandas y las compila en estructuras JSON estructuradas que alimentan el Vector Store local.

C. Módulo RAG (Retrieval-Augmented Generation - Capa de Conocimiento)

Este módulo dota al agente de una base de conocimiento a largo plazo sobre el contexto del usuario.

- Indexador/Embedding: Segmenta en fragmentos estandarizados (chunks) los manuales de RRHH, cultura de la empresa, guías de configuración y los Learning Paths técnicos (cursos de Git, arquitecturas y tecnologías para perfiles Junior). Transforma el texto en vectores densos de 1536 dimensiones mediante el modelo text-embedding-3-small.
- Base de Datos Vectorial (Vector Store): Almacena y persiste localmente los vectores documentales y de perfil dentro de colecciones aisladas en ChromaDB.
- Componente de Recuperación (Retriever): Expone las interfaces semánticas necesarias para buscar por similitud de coseno y recuperar exclusivamente los fragmentos informativos que el cerebro del agente requiera en su fase de razonamiento.

D. Módulo de Planificación Multi-Step y Orquestación (Capa de Razonamiento)

Representa el núcleo cognitivo centralizado del agente. Implementa una estrategia avanzada del framework ReAct (Reasoning and Acting) montada sobre un grafo dirigido de LangGraph:

- Submódulo Planificador (Plan & Solve): Ante solicitudes multitarea o de alta ambigüedad, el LLM frena la ejecución e hilvana un plan lógico de subtareas secuenciales antes de interactuar con el entorno.
- Evaluador de Restricciones Cruzadas: Módulo lógico que contrasta de manera matemática las normativas organizacionales y preferencias de rutina recuperadas semánticamente vía RAG frente al estado real de ocupación de las franjas horarias devueltas por el calendario.

- Gestor de Memoria conversacional: Actúa como un reductor de estado que indexa y conserva los últimos turnos de mensajes de la sesión actual en la ventana de contexto del prompt local, resolviendo correcciones operativas y referencias cruzadas en tiempo real.

E. Módulo de Integración y Actuadores (Capa de Ejecución)

El componente instrumental encargado de acoplar de manera estricta el pensamiento abstracto del modelo con las modificaciones del entorno digital.

- Mecanismo de Function Calling: Valida los argumentos lógicos generados por el orquestador central y los parsea a esquemas estructurados JSON estrictos para cumplir con las firmas del catálogo de funciones.
- Conector de API Extrínseca: Administra la comunicación segura (OAuth2) con la infraestructura REST de Google Calendar, capturando excepciones de red para transformarlas en observaciones textuales digeribles por el bucle ReAct.

2. Catálogo de Componentes Tecnológicos (Stack Tecnológico)

- Orquestación del Grafo de Estados: LangGraph / LangChain (Tier 1).
- Motor de Razonamiento Base (LLM): OpenAI GPT-4o-mini.
- Motor de Transcripción de Audio: OpenAI Whisper API.
- Base de Datos Vectorial (RAG): ChromaDB (Local Store).
- Capa de Embeddings: OpenAI text-embedding-3-small.
- Entorno y API Externa: Google Calendar API.
- Canal de Interfaz: Twilio API for WhatsApp / Flask.

3. Catálogo de Herramientas y Acciones (Actuadores del Sistema)

Como consecuencia del nuevo modelo híbrido, el catálogo de funciones (Tools) que el orquestador central puede invocar de forma dinámica mediante Function Calling se amplía, quedando formalizado de la siguiente manera:

El catálogo de funciones funcionales que el orquestador central puede invocar mediante Function Calling queda formalizado técnicamente de la siguiente manera:

1. query_company_knowledge(query: str) -> str: Realiza una búsqueda de similitud semántica en el Vector Store local (ChromaDB) sobre el corpus normativo de la empresa. Extrae de forma exacta fragmentos informativos sobre manuales de RRHH, lineamientos de cultura organizacional y guías de las rutas de aprendizaje tecnológico asignadas.
2. query_user_profile(query: str) -> str: Recupera y procesa las restricciones semánticas y hábitos personales de la rutina del usuario recolectados durante la entrevista inicial de la Fase 1, transformando la consulta en embeddings para su emparejamiento.

3. `get_calendar_events(start_time: str, end_time: str) -> list`: Conecta de manera determinística con la API de Google Calendar y retorna una lista de objetos JSON con los eventos y compromisos previamente agendados entre las marcas de tiempo ISO especificadas.
4. `insert_calendar_event(title: str, start_time: str, end_time: str, description: str) -> dict`: Modifica físicamente el entorno externo insertando un nuevo bloque temporal en la agenda del empleado (bloques de estudio de tecnologías de la ruta de capacitación, sesiones de mentoría con líderes, etc.).
5. `find_available_slots(date: str, duration_minutes: int) -> list`: Herramienta algorítmica auxiliar que procesa vectorialmente una fecha, analiza la agenda en busca de colisiones horarias y devuelve las franjas de tiempo libres óptimas que cumplen con la duración requerida.

Este diseño arquitectónico garantiza la separación de responsabilidades, permitiendo al agente desglosar planes complejos paso a paso (Multi-Step), contextualizado bajo las necesidades únicas del empleado o profesional gracias a la inyección dinámica de conocimiento por RAG.

Descripción del Flujo de Interacción:

1. Input del Usuario: El ingresante inicia el turno enviando un mensaje a la interfaz de WhatsApp. El sistema discrimina el tipo de input; ante una percepción de audio, delega la señal a la API de Whisper para unificar el estímulo en una cadena de texto plano.
2. Actualizar Memoria y Generar Prompt: El orquestador actualiza el historial multi-turno de la conversación y ensambla un prompt estructurado inyectando: las instrucciones del sistema, las descripciones explícitas del catálogo de Tools y las variables contextuales vigentes. El prompt consolidado se despacha hacia el LLM (gpt-4o-mini).
3. Procesar LLM y Bucle ReAct: El modelo de lenguaje procesa las entradas semánticas y genera un pensamiento interno (Thought). Si determina que requiere información normativa o alterar la agenda externa para cumplir sus objetivos, emite un bloque estructurado de ejecución (Tool Call) iniciando formalmente el bucle de razonamiento.
4. Bucle ReAct / Multi-Step (Ejecución): La capa de integración intercepta la petición del LLM, valida los parámetros y despacha la acción según corresponda:
 - Si el plan cognitivo exige conocer qué capacitación requiere un perfil Junior o políticas corporativas, ejecuta la herramienta de RAG `query_company_knowledge`.
 - Si necesita contrastar las preferencias de horarios del usuario contra la realidad de su calendario, invoca en paralelo `query_user_profile` y `get_calendar_events`.

5. Observación y Re-entrada en Bucle: El resultado físico de la función ejecutada se captura, se encapsula bajo el rol de una Observación semántica y se inyecta de regreso al prompt base del agente. El flujo retorna al paso de generación de prompt, donde el LLM evalúa si se han completado los múltiples pasos de la tarea (Multi-Step Planning) o si requiere disparar otra herramienta del catálogo.
6. Multi-Step Finalizado y Respuesta Final: Cuando el modelo de lenguaje concluye que el plan está resuelto y no requiere más interacciones instrumentales, procesa una salida final sintetizada en lenguaje natural amigable. La capa de interfaz transmite este mensaje de confirmación al empleado por WhatsApp, cerrando el ciclo transaccional con la instrucción implícita del término 'END'.

Avances en la Implementación (Segunda Entrega):

La fase de implementación materializa el diseño conceptual mediante un ecosistema de tecnologías basadas en Python. Se adoptó el framework LangGraph para estructurar el flujo de control del agente, lo que permite un manejo explícito del estado y facilita la implementación de arquitecturas cíclicas como el patrón ReAct.

1. Implementación base del agente y Flujo de interacción con el LLM

El núcleo del agente se implementa instanciando un Modelo de Lenguaje de Gran Escala (LLM), específicamente GROQ. Este modelo fue seleccionado por su alta fiabilidad en el uso de function calling y su baja latencia.

El flujo de interacción comienza cuando el sistema recibe el input del usuario. Este mensaje se anexa a una lista de mensajes y se envía al LLM junto con un System Prompt robusto. Este prompt condiciona el comportamiento del modelo, estableciendo su "persona" y sus reglas operativas inquebrantables:

"Eres un Mentor y Asistente de Onboarding corporativo. Tus directivas estrictas son: 1) Nunca inventes reglas de la empresa ni planes de estudio; utiliza siempre la herramienta de búsqueda en la base de conocimiento para consultar manuales y rutas de aprendizaje. 2) Antes de agendar bloques de capacitación o mentorías, es obligatorio que consultes la disponibilidad en el calendario del usuario..."

Las herramientas (Tools) se vinculan directamente al modelo mediante el método `.bind_tools(tools)`, lo que habilita al LLM para devolver respuestas estructuradas indicando qué función desea ejecutar y con qué parámetros.

2. Implementación del módulo de contexto y manejo del ambiente

El agente opera en un entorno parcialmente observable (el calendario y los manuales). Para manejar este entorno y mantener la coherencia, se implementa un

Módulo de Contexto basado en el manejo de estado de LangGraph (MessagesState).

- Memoria de Corto Plazo: El estado del grafo conserva el historial de la conversación actual como una secuencia de objetos HumanMessage, AIMessage y ToolMessage. Esto permite interacciones multi-turno fluidas. Si el agente propone "Tengo un espacio libre a las 10:00 o a las 15:00", y el usuario responde "Anotame en el primero", el LLM puede resolver la referencia anafórica gracias a esta ventana de contexto.
- Manejo del Ambiente: El ambiente se percibe de forma retardada a través de las "Observaciones". Cuando el agente interactúa con el entorno (ej. intentando leer la agenda), el resultado se encapsula en un ToolMessage y se inyecta en el estado, actualizando la visión del mundo que tiene el LLM.

3. Conexiones externas necesarias (APIs, base de datos)

El sistema depende de tres pilares externos fundamentales:

1. Google Calendar API (REST): Conexión vía OAuth2. Se consumen los endpoints events.list (para lectura determinística de franjas horarias ocupadas) y events.insert (para la escritura de nuevos bloques temporales).
2. ChromaDB (Vector Store Local): Base de datos vectorial embebida que actúa como la memoria a largo plazo de la empresa. Almacena los manuales de empleado y planes de carrera.
3. OpenAI API: Provee los endpoints para inferencia del modelo generativo (gpt-4o-mini) y para el modelo de vectorización (text-embedding-3-small), el cual traduce los textos corporativos a vectores de 1536 dimensiones.

4. Integración de herramientas (Tools) y fuentes externas (RAG)

Las herramientas se implementan como funciones en Python decoradas con @tool de LangChain. Un aspecto crítico de este diseño es que el RAG no es un proceso oculto, sino una herramienta explícita.

- Integración RAG (query_company_knowledge): Cuando el LLM invoca esta herramienta, el código recibe la query (ej. "tareas del primer día"), la convierte en un embedding, y realiza una búsqueda de similitud por coseno en ChromaDB. Retorna los fragmentos de texto (chunks) más relevantes del manual, inyectándolos como evidencia dura para que el LLM responda sin alucinar.
- Integración de Agenda: Herramientas como get_calendar_events y insert_calendar_event transforman las peticiones en lenguaje natural en llamadas GET y POST estructuradas en JSON hacia los servidores de Google.

5. Workflow y Loop / Estrategia de razonamiento del agente

El Workflow se define mediante un grafo de estados dirigido (StateGraph). Se diseñó un ciclo de enrutamiento condicional (Bucle ReAct) que intercala el razonamiento semántico con acciones instrumentales:

1. **Nodo agent:** Invoca al LLM con el estado actual. Si el LLM decide que necesita información, emite un Tool Call. Si determina que ya cumplió su objetivo, emite la respuesta final.
2. **Arista Condicional:** Evalúa la salida del nodo agent. Si hay llamadas a herramientas, rutea al nodo tools; de lo contrario, rutea a END.
3. **Nodo tools (ToolNode):** Ejecuta la función física de Python solicitada (consultar RAG, leer agenda, etc.), captura el retorno, lo anexa al estado y devuelve el flujo al nodo agent para que reevalúe.

Ejemplo de estrategia de razonamiento Multi-Step:

- **Pensamiento interno:** "El usuario debe iniciar su onboarding. Paso 1: Consultar RAG para saber qué curso le toca. Paso 2: Leer el calendario para buscar huecos libres. Paso 3: Agendar el curso."
- **Bucle Iterativo:** El grafo itera de forma autónoma pasando del nodo agent al nodo tools múltiples veces, resolviendo cada sub-tarea, hasta que el LLM considera que la rutina del ingresante está completamente organizada.

6. Logging básico de las llamadas al LLM y Tools

Para garantizar la observabilidad del sistema y facilitar la depuración (debugging), se implementa un registro de eventos detallado. Se utilizan Callbacks propios del framework y la librería estándar logging de Python para imprimir por consola (stdout) y volcar en un archivo temporal cada evento crítico:

- **Entrada/Salida de Nodos:** Se registra el inicio y fin de la ejecución del nodo del agente.
- **Trazabilidad de Tools:** Cada vez que el LLM invoca una herramienta, se imprime el nombre de la función y el payload exacto de argumentos (ej. [CALL] get_calendar_events | args: {"start_time": "2026-06-15...", "end_time": "..."}).
- **Consumo de Tokens:** Se registra el costo computacional de cada iteración (tokens de entrada de la memoria de contexto y tokens de salida), métrica vital para analizar la viabilidad económica del agente en un entorno de producción.